

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2
По дисциплине «Интеллектуальный анализ данных»
Тема: «Автоэнкодеры»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);

2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;

4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;

5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
17	Mushroom	poisonous

Ход работы

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.manifold import TSNE
from sklearn.model_selection import train_test_split

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
torch.manual_seed(RANDOM_STATE)

try:
    from ucimlrepo import fetch_ucirepo
    mushroom = fetch_ucirepo(id=73)
    X_raw = mushroom.data.features
    y_raw = mushroom.data.targets
    df = pd.concat([X_raw, y_raw], axis=1)
except Exception as e:
    print("Не удалось скачать через ucimlrepo, читаем локальный файл:", e)
    columns = [
        "poisonous", "cap-shape", "cap-surface", "cap-color", "bruises", "odor",
        "gill-attachment", "gill-spacing", "gill-size", "gill-color",
        "stalk-shape", "stalk-root", "stalk-surface-above-ring",
        "stalk-surface-below-ring", "stalk-color-above-ring",
        "stalk-color-below-ring", "veil-type", "veil-color", "ring-number",
        "ring-type", "spore-print-color", "population", "habitat"
    ]
    df = pd.read_csv("agaricus-lepiota.data", names=columns)

df = df.replace("?", np.nan).dropna()

target_col = "poisonous"
y = df[target_col].values
```

```

X_cat = df.drop(columns=[target_col])

encoder = OneHotEncoder(sparse_output=False)
X_encoded = encoder.fit_transform(X_cat)

le = LabelEncoder()
y_encoded = le.fit_transform(y)
class_names = le.classes_

print("Форма закодированной выборки:", X_encoded.shape)
print("Классы:", class_names)

X_data = X_encoded.astype(np.float32)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 64),
            nn.ReLU(),
            nn.Linear(64, input_dim),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        x_rec = self.decoder(z)
        return x_rec, z

def train_autoencoder(X, latent_dim, epochs=60, batch_size=64, lr=1e-3):
    input_dim = X.shape[1]
    model = Autoencoder(input_dim, latent_dim).to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss()

    X_tensor = torch.tensor(X, dtype=torch.float32)
    dataset = TensorDataset(X_tensor)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    model.train()
    loss_history = []

    for epoch in range(epochs):
        epoch_loss = 0.0

        for (batch,) in loader:
            batch = batch.to(device)
            optimizer.zero_grad()
            recon, _ = model(batch)
            loss = criterion(recon, batch)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item() * batch.size(0)

        epoch_loss /= len(dataset)
        loss_history.append(epoch_loss)

```

```

        if (epoch + 1) % 10 == 0 or epoch == 0:
            print(f"[latent_dim={latent_dim}] Эпоха {epoch+1}/{epochs}, MSE loss: {epoch_loss:.5f}")

    model.eval()

    with torch.no_grad():
        _, Z = model(X_tensor.to(device))

    Z = Z.cpu().numpy()

    return model, Z, loss_history

print("\n--- Обучение автоэнкодера (2 нейрона в среднем слое) ---")
ae_model_2d, Z_ae_2d, loss_hist_2d = train_autoencoder(X_data, latent_dim=2)

print("\n--- Обучение автоэнкодера (3 нейрона в среднем слое) ---")
ae_model_3d, Z_ae_3d, loss_hist_3d = train_autoencoder(X_data, latent_dim=3)

plt.figure(figsize=(8, 5))
plt.plot(loss_hist_2d, label="Автоэнкодер (latent=2)")
plt.plot(loss_hist_3d, label="Автоэнкодер (latent=3)")
plt.xlabel("Эпоха")
plt.ylabel("MSE loss")
plt.title("Сходимость обучения автоэнкодера")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("ae_training_loss.png", dpi=150)
plt.show()

class MyPCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.mean_ = None
        self.components_ = None
        self.explained_variance_ratio_ = None

    def fit(self, X):
        self.mean_ = np.mean(X, axis=0)
        X_centered = X - self.mean_

        cov_matrix = np.cov(X_centered, rowvar=False)

        eig_values, eig_vectors = np.linalg.eigh(cov_matrix)

        idx = np.argsort(eig_values)[::-1]
        eig_values = eig_values[idx]
        eig_vectors = eig_vectors[:, idx]

        self.components_ = eig_vectors[:, :self.n_components]
        total_var = np.sum(eig_values)
        self.explained_variance_ratio_ = eig_values[:self.n_components] / total_var

        return self

    def transform(self, X):
        X_centered = X - self.mean_
        return np.dot(X_centered, self.components_)

    def fit_transform(self, X):
        self.fit(X)
        return self.transform(X)

print("\n--- Применение собственного PCA (2 компоненты) ---")
pca2 = MyPCA(n_components=2)
Z_pca_2d = pca2.fit_transform(X_data)
print(

```

```

        "Объясненная дисперсия (2 комп.):",
        pca2.explained_variance_ratio_,
        "сумма:",
        pca2.explained_variance_ratio_.sum()
    )

    print("\n--- Применение собственного PCA (3 компоненты) ---")
    pca3 = MyPCA(n_components=3)
    Z_pca_3d = pca3.fit_transform(X_data)
    print(
        "Объясненная дисперсия (3 комп.):",
        pca3.explained_variance_ratio_,
        "сумма:",
        pca3.explained_variance_ratio_.sum()
    )

    N_SUBSAMPLE = 3000

    if X_data.shape[0] > N_SUBSAMPLE:
        idx_sub = np.random.choice(X_data.shape[0], N_SUBSAMPLE, replace=False)
    else:
        idx_sub = np.arange(X_data.shape[0])

    X_sub = X_data[idx_sub]
    y_sub = y_encoded[idx_sub]

    perplexities = [20, 30, 40, 50, 60]

    tsne_results_2d = {}

    for perp in perplexities:
        print(f"\nt-SNE (2D), perplexity={perp} ...")
        tsne = TSNE(
            n_components=2,
            perplexity=perp,
            init="pca",
            random_state=RANDOM_STATE,
            learning_rate="auto"
        )
        tsne_results_2d[perp] = tsne.fit_transform(X_sub)

    tsne_results_3d = {}

    for perp in perplexities:
        print(f"t-SNE (3D), perplexity={perp} ...")
        tsne = TSNE(
            n_components=3,
            perplexity=perp,
            init="pca",
            random_state=RANDOM_STATE,
            learning_rate="auto"
        )
        tsne_results_3d[perp] = tsne.fit_transform(X_sub)

    colors = {0: "tab:green", 1: "tab:red"}
    labels_map = {0: class_names[0], 1: class_names[1]}

    def plot_2d(Z, y, title, filename):
        plt.figure(figsize=(7, 6))

        for cls in np.unique(y):
            mask = y == cls
            plt.scatter(
                Z[mask, 0],
                Z[mask, 1],
                s=8,
                alpha=0.6,
                c=colors[cls],

```

```

        label=labels_map[cls]
    )

    plt.xlabel("Компонента 1")
    plt.ylabel("Компонента 2")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.show()

def plot_3d(Z, y, title, filename):
    fig = plt.figure(figsize=(8, 7))
    ax = fig.add_subplot(111, projection="3d")

    for cls in np.unique(y):
        mask = y == cls
        ax.scatter(
            Z[mask, 0],
            Z[mask, 1],
            Z[mask, 2],
            s=8,
            alpha=0.6,
            c=colors[cls],
            label=labels_map[cls]
        )

    ax.set_xlabel("Компонента 1")
    ax.set_ylabel("Компонента 2")
    ax.set_zlabel("Компонента 3")
    ax.set_title(title)
    ax.legend()

    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.show()

plot_2d(
    Z_ae_2d,
    y_encoded,
    "Автоэнкодер, 2 компоненты",
    "ae_2d.png"
)

plot_3d(
    Z_ae_3d,
    y_encoded,
    "Автоэнкодер, 3 компоненты",
    "ae_3d.png"
)

plot_2d(
    Z_pca_2d,
    y_encoded,
    "PCA (собственная реализация), 2 компоненты",
    "pca_2d.png"
)

plot_3d(
    Z_pca_3d,
    y_encoded,
    "PCA (собственная реализация), 3 компоненты",
    "pca_3d.png"
)

for perp in perplexities:
    plot_2d(

```

```

        tsne_results_2d[perp],
        y_sub,
        f"t-SNE, 2 компоненты, perplexity={perp}",
        f"tsne_2d_perp{perp}.png"
    )

for perp in perplexities:
    plot_3d(
        tsne_results_3d[perp],
        y_sub,
        f"t-SNE, 3 компоненты, perplexity={perp}",
        f"tsne_3d_perp{perp}.png"
    )

print("\n=== Сводка ===")
print(
    f"Размер выборки после очистки: {X_data.shape[0]} объектов, "
    f"{X_data.shape[1]} признаков (после ONE)"
)
print(f"Автоэнкодер (2D) финальный loss: {loss_hist_2d[-1]:.5f}")
print(f"Автоэнкодер (3D) финальный loss: {loss_hist_3d[-1]:.5f}")
print(
    f"PCA (2 комп.) объясненная дисперсия: "
    f"{pca2.explained_variance_ratio_.sum():.4f}"
)
print(
    f"PCA (3 комп.) объясненная дисперсия: "
    f"{pca3.explained_variance_ratio_.sum():.4f}"
)

```

Вывод программы:

```

Форма закодированной выборки: (5644, 98)
Классы: ['e' 'p']

--- Обучение автоэнкодера (2 нейрона в среднем слое) ---
[latent_dim=2] Эпоха 1/60, MSE loss: 0.16658
[latent_dim=2] Эпоха 10/60, MSE loss: 0.05230
[latent_dim=2] Эпоха 20/60, MSE loss: 0.04605
[latent_dim=2] Эпоха 30/60, MSE loss: 0.04330
[latent_dim=2] Эпоха 40/60, MSE loss: 0.04086
[latent_dim=2] Эпоха 50/60, MSE loss: 0.03942
[latent_dim=2] Эпоха 60/60, MSE loss: 0.03862

--- Обучение автоэнкодера (3 нейрона в среднем слое) ---
[latent_dim=3] Эпоха 1/60, MSE loss: 0.15109
[latent_dim=3] Эпоха 10/60, MSE loss: 0.04649
[latent_dim=3] Эпоха 20/60, MSE loss: 0.04054
[latent_dim=3] Эпоха 30/60, MSE loss: 0.03715
[latent_dim=3] Эпоха 40/60, MSE loss: 0.03520
[latent_dim=3] Эпоха 50/60, MSE loss: 0.03391
[latent_dim=3] Эпоха 60/60, MSE loss: 0.03243

--- Применение собственного PCA (2 компоненты) ---
Объясненная дисперсия (2 комп.): [0.22202824 0.13786022] сумма: 0.3598884546108805

--- Применение собственного PCA (3 компоненты) ---
Объясненная дисперсия (3 комп.): [0.22202824 0.13786022 0.09496084] сумма: 0.4548492903213879

```

```
t-SNE (2D), perplexity=20 ...
```

```
t-SNE (2D), perplexity=30 ...
```

```
t-SNE (2D), perplexity=40 ...
```

```
t-SNE (2D), perplexity=50 ...
```

```
t-SNE (2D), perplexity=60 ...
```

```
t-SNE (3D), perplexity=20 ...
```

```
t-SNE (3D), perplexity=30 ...
```

```
t-SNE (3D), perplexity=40 ...
```

```
t-SNE (3D), perplexity=50 ...
```

```
t-SNE (3D), perplexity=60 ...
```

```
=== Сводка ===
```

```
Размер выборки после очистки: 5644 объектов, 98 признаков (после ONE)
```

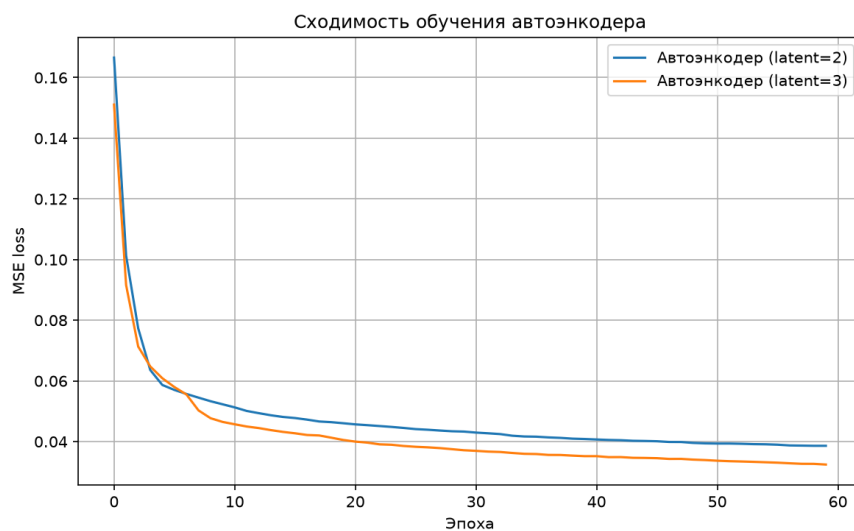
```
Автоэнкодер (2D) финальный loss: 0.03862
```

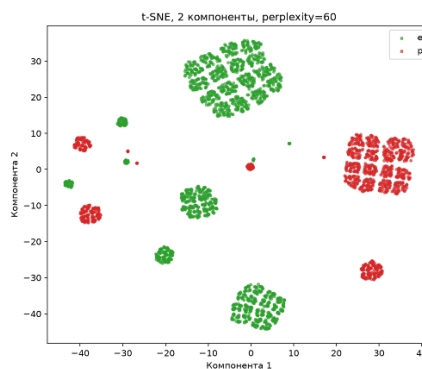
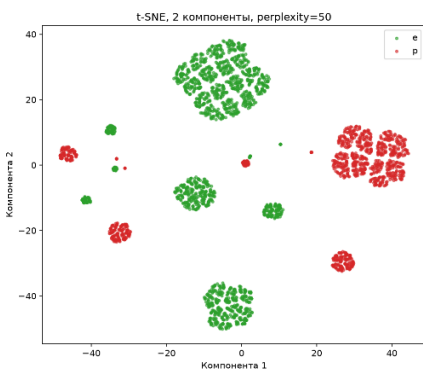
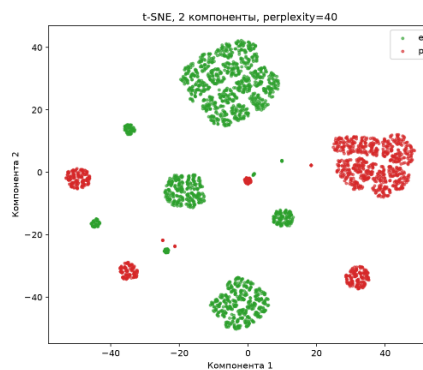
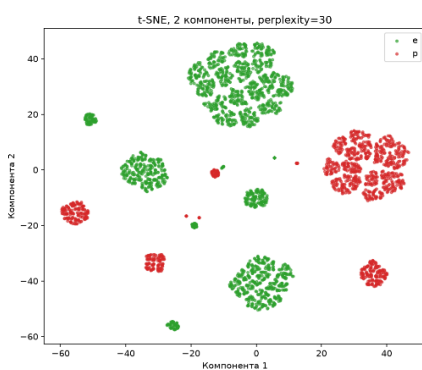
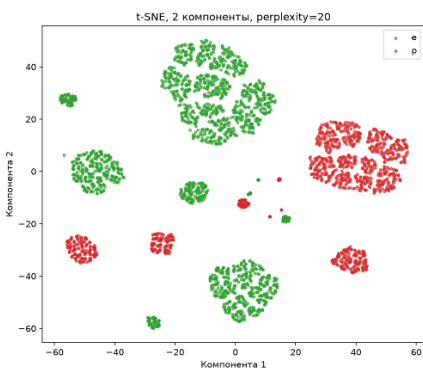
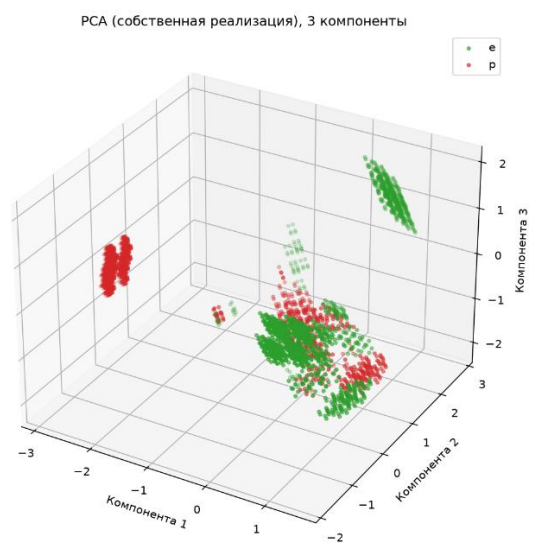
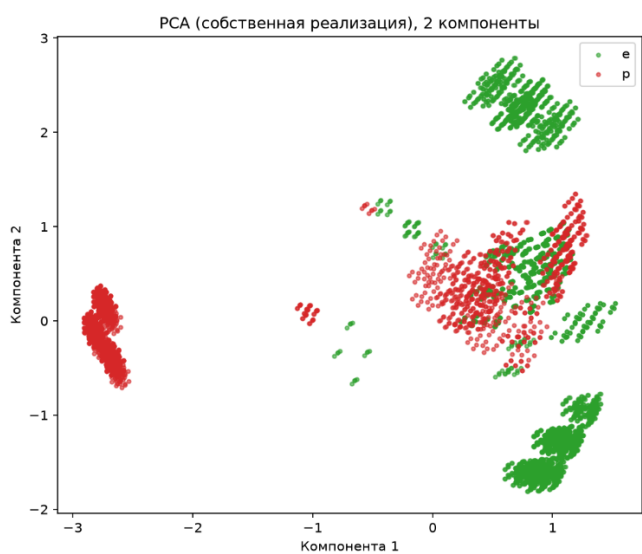
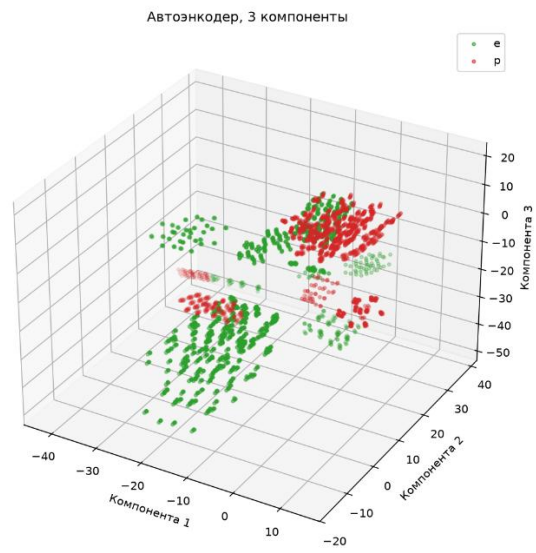
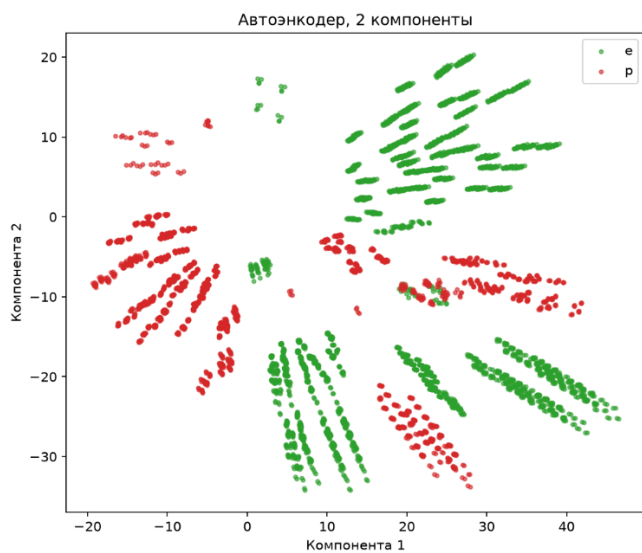
```
Автоэнкодер (3D) финальный loss: 0.03243
```

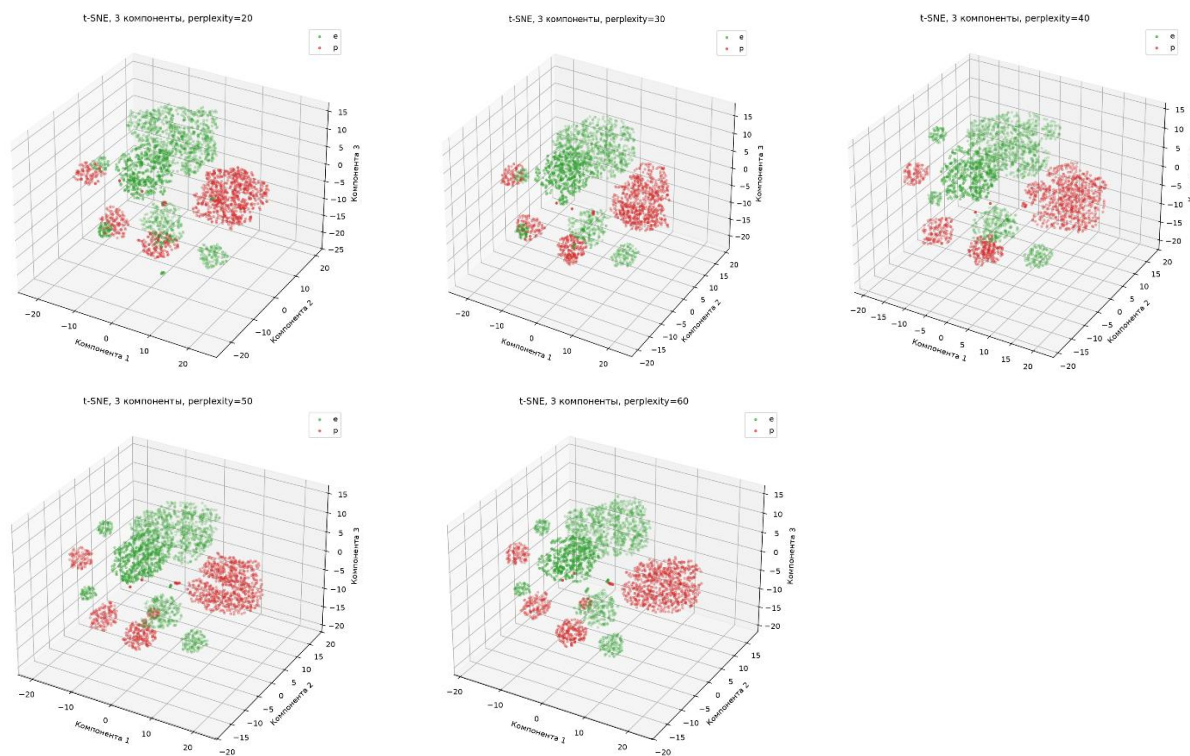
```
PCA (2 комп.) объясненная дисперсия: 0.3599
```

```
PCA (3 комп.) объясненная дисперсия: 0.4548
```

Графики:







В ходе лабораторной работы к датасету Mushrooms, содержащему 5644 объекта и 98 признаков после кодирования, были применены три метода снижения размерности: автоэнкодер, PCA и t-SNE. Автоэнкодер с архитектурой $98 \rightarrow 64 \rightarrow 32 \rightarrow 2/3$, обученный с оптимизатором Adam и функцией потерь MSE, позволил получить двухмерное и трёхмерное представления данных. При двух компонентах итоговая ошибка составила 0,03862, а при трёх компонентах 0,03243, что показывает более точное восстановление данных при увеличении размерности скрытого пространства. Метод PCA, реализованный вручную через `numpy.linalg.eigh`, сохранил 35,99% исходной дисперсии при переходе к двум компонентам и 45,48% при переходе к трём компонентам. Метод t-SNE был исследован при значениях perplexity от 20 до 60, что позволило получить различные варианты двумерного и трёхмерного представления классов. Проведённое сравнение показало различия в характере получаемых представлений, при этом автоэнкодер позволяет гибко формировать латентное пространство, PCA сохраняет значительную часть глобальной структуры данных, а t-SNE предназначен преимущественно для визуального анализа структуры и разделения классов.

Вывод: научилась применять автоэнкодеры для осуществления визуализации данных и их анализа.